Digital Logic Circuit Design

VHDL Assignment-2

조합논리회로 구현 및 시뮬레이션

Logisim-evolution VHDL Entity를 이용한 회로 구현

학과 로봇공학과

학번 2023050146

이름 김태수

구현 항목

1. 4입력 AND 및 OR
2. 8-to-1 Multiplexer
3. 7-Segment 문자 표시
4. 25인 투표기
5. 13비트 입력의 1 개수 표시

실습 자료 VHDL_1의 1~5번 과제 구현 및 시뮬레이션 결과

실험 목적과 구성

VHDL 소스, Logisim 회로, 실행 결과

1. 실험 목적

VHDL로 4입력 AND/OR, 8-to-1 MUX, 7-Segment 문자 표시기, 25인 투표기, 13비트 개수 표시기를 구현하였다.
Logisim-evolution v5.0.0에서 VHDL Entity 블록을 배치하고 입력값을 바꾸어 출력 동작을 확인하였다.

2. 구현 개요

문제	실제 Logisim Entity	핵심 동작
1	logic_test	a,b,c,d → AND / OR
2	mux8_1	8개 입력과 3비트 선택 신호
3	seg7_letters	sel(2:0) → display(7:0)
4	vote25	25비트 찬성표 합계 ≥ 13
5	counter13_seg7	13비트 1 개수 + enable + 7-Segment

3. 검증 및 표기 기준

각 회로에서 주요 입력 조합과 경계 조건을 선택하여 결과를 확인하였다. 7-Segment 출력은 a,b,c,d,e,f,g,dp 순서의 8비트이며, 1인 선분이 켜지는 방식으로 연결하였다.

과제 요구사항에 따라 다섯 개의 회로를 구성하였으며, 실험 결과에는 제출한 VHDL 소스와 Logisim 캡처에서 확인할 수 있는 사례만 기록하였다.

4입력 AND 및 OR

VHDL Entity logic_test / a,b,c,d → out1,out2

VHDL 코드 (Entity: logic_test)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

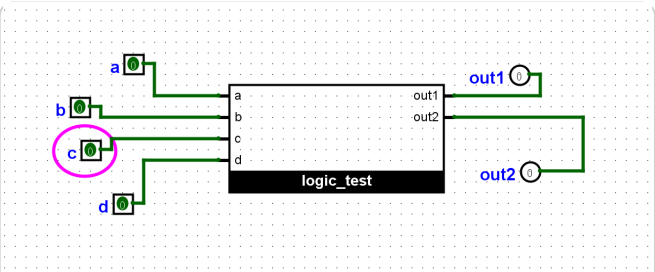
entity logic_test is
    port (
        a, b, c, d : in STD_LOGIC;
        out1, out2 : out STD_LOGIC
    );
end logic_test;

architecture Behavioral of logic_test is
begin
    out1 <= a and b and c and d;
    out2 <= a or b or c or d;
end Behavioral;
```

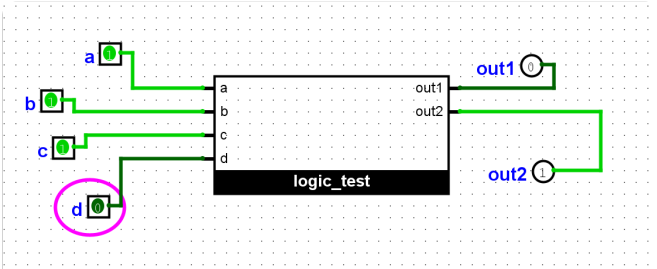
입출력 검증

입력 abcd	out1 (AND)	out2 (OR)	관찰
0000	0	0	정상
1110	0	1	정상
1111	1	1	정상

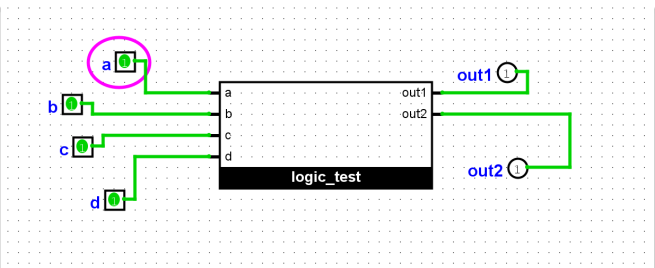
Logisim 실행 캡처



입력 0000 → 0, 0



입력 1110 → 0, 1



입력 1111 → 1, 1

8-to-1 Multiplexer

VHDL Entity mux8_1 / a~h, ch[2:0] → data

VHDL 코드

```
library ieee;
use ieee.std_logic_1164.all;

entity mux8_1 is
    port(
        a, b, c, d : in std_logic;
        e, f, g, h : in std_logic;
        ch : in std_logic_vector(2 downto 0);
        data : out std_logic
    );
end entity mux8_1;

architecture bhv of mux8_1 is
begin
    process(a,b,c,d,e,f,g,h,ch)
    begin
        case ch is
            when "000" => data <= a;
            when "001" => data <= b;
            when "010" => data <= c;
            when "011" => data <= d;
            when "100" => data <= e;
            when "101" => data <= f;
            when "110" => data <= g;
            when "111" => data <= h;
            when others => data <= '0';
        end case;
    end process;
end architecture bhv;
```

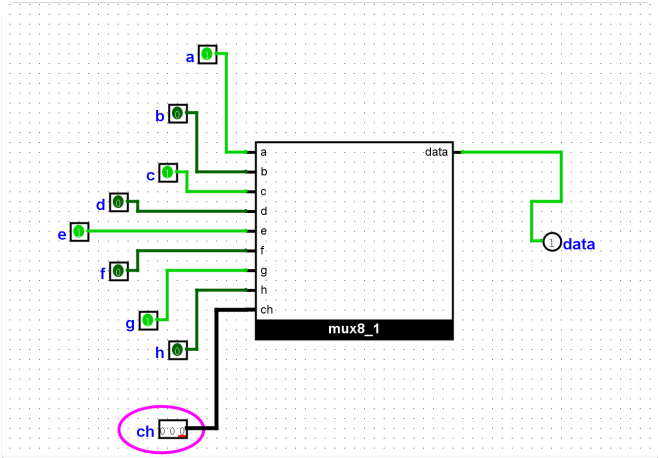
회로 동작 원리

선택 신호 ch가 000~111 사이에서 변화하면 입력 a,b,c,d,e,f,g,h 중 해당 위치의 값을 data로 전달한다. 테스트에서는 a,c,e,g를 1, b,d,f,h를 0으로 설정하고 선택 신호를 바꿔 확인하였다.

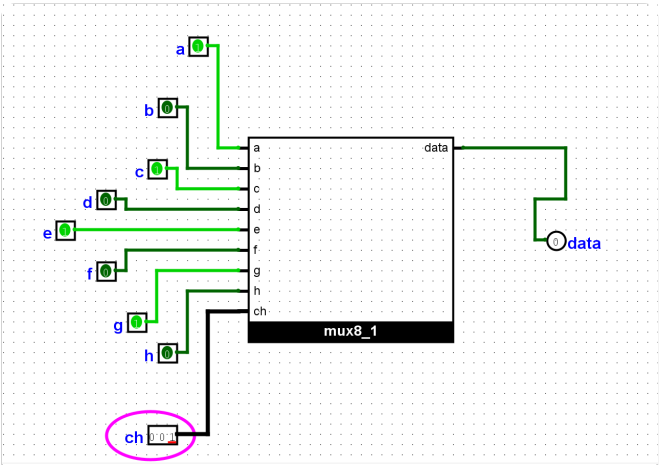
ch	선택 입력	관찰 출력 data
000	a=1	1
001	b=0	0
110	g=1	1
111	h=0	0

MUX 동작 캡처

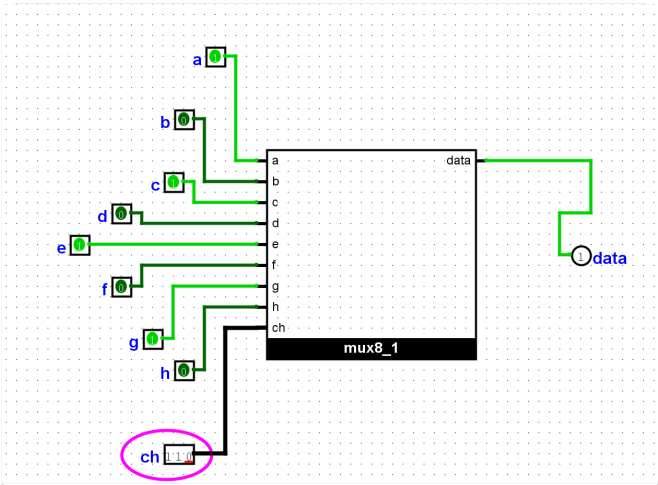
네모난 VHDL 블록과 3비트 ch 신호를 이용한 선택 기능 확인



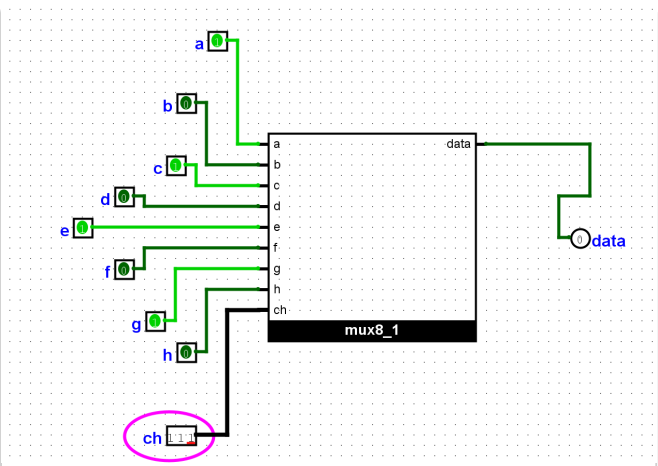
ch=000 → a=1



ch=001 → b=0



ch=110 → g=1



ch=111 → h=0

네 가지 대표 입력에서 출력이 선택된 입력 신호와 일치하였다. 작성한 case 문은 ch의 8가지 유효한 이진 조합을 각각 처리하며, 나머지 경우에는 0을 할당한다.

7-Segment 문자 표시기

VHDL Entity seg7_letters / 3비트 sel로 문자 선택

VHDL 코드

```
library ieee;
use ieee.std_logic_1164.all;

entity seg7_letters is
    port(
        sel : in std_logic_vector(2 downto 0);
        display : out std_logic_vector(7 downto 0)
    );
end entity seg7_letters;

architecture bhv of seg7_letters is
begin
    process(sel)
    begin
        case sel is
            when "000" => display <= "11111010"; -- a
            when "001" => display <= "00111110"; -- b
            when "010" => display <= "00011010"; -- c
            when "011" => display <= "01111010"; -- d
            when "100" => display <= "10001110"; -- f
            when "101" => display <= "11110110"; -- g
            when others => display <= "00000000";
        end case;
    end process;
end architecture bhv;
```

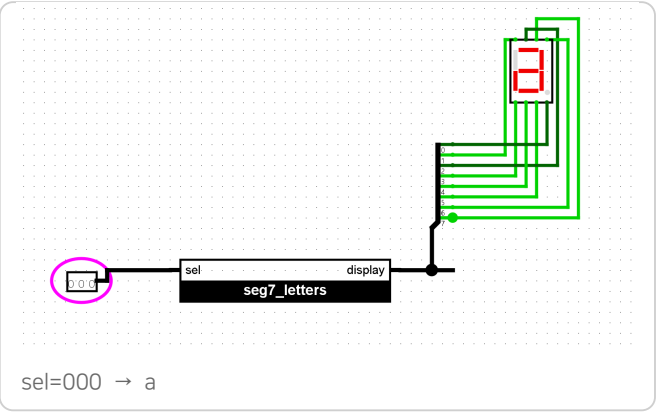
선택 신호와 출력 패턴

sel	표시 문자	8비트 출력 (a~g,dp)
000	a	11111010
001	b	00111110
010	c	00011010
011	d	01111010
100	f	10001110
101	g	11110110
110/111	꺼짐	00000000

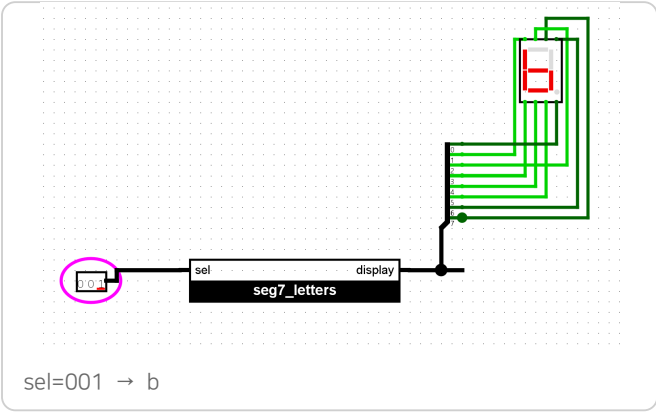
과제에서 지정한 소문자 a, b, c, d, f, g를 3비트 선택 신호에 순서대로 배정하였다. 7-Segment는 일반 글꼴처럼 모든 획을 표현할 수 없으므로, 소스 코드의 출력 패턴에 대응하는 선분 조합으로 문자를 표시하였다.

문자 선택 시뮬레이션

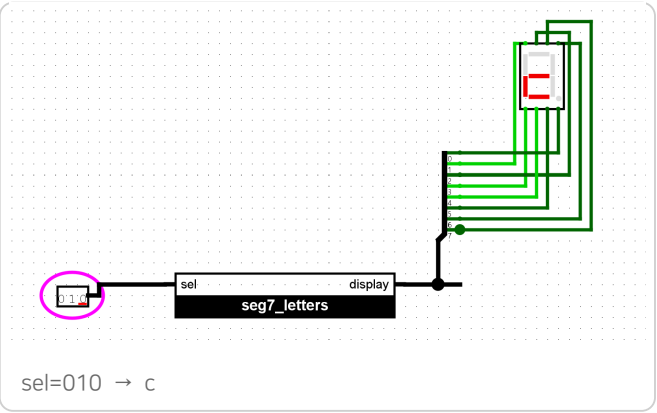
VHDL Entity + Splitter + 7-Segment



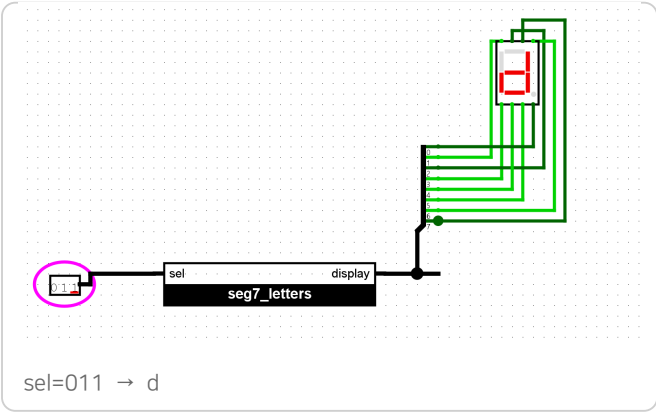
sel=000 → a



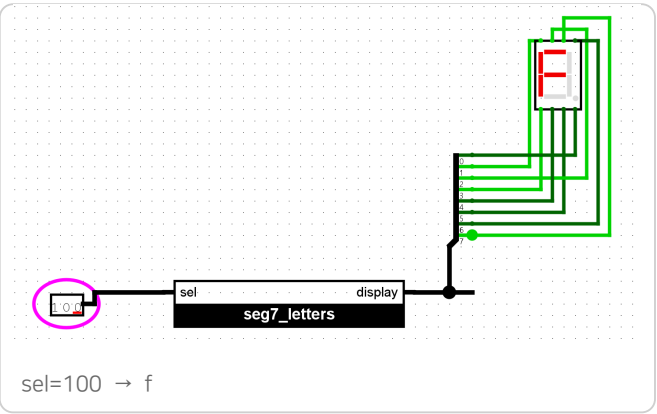
sel=001 → b



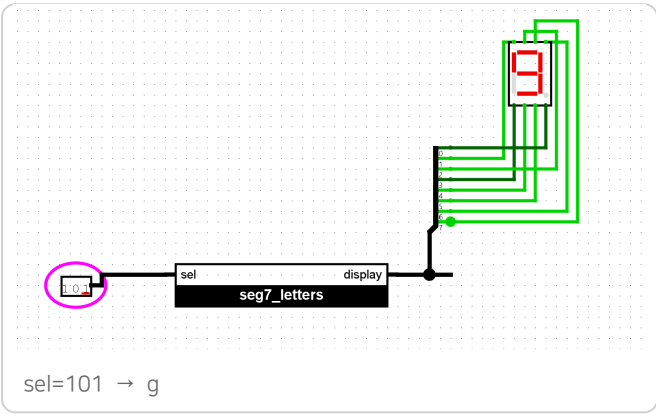
sel=010 → c



sel=011 → d



sel=100 → f



sel=101 → g

sel을 000~101로 변경했을 때 지정된 여섯 문자에 대응하는 선분 패턴이 나타났다. sel이 110 또는 111이면 모든 선분을 끄도록 작성하였다.

25인 투표기

VHDL Entity vote25 / 찬성 수가 13 이상일 때 result=1

VHDL 소스 (vote25.vhd)

```
library ieee;
use ieee.std_logic_1164.all;

entity vote25 is
    port(
        vt : in std_logic_vector(24 downto 0);
        result : out std_logic
    );
end entity vote25;

architecture bhv of vote25 is
begin
    process(vt)
        variable sum : integer range 0 to 25;
    begin
        sum := 0;

        for i in 0 to 24 loop
            if vt(i) = '1' then
                sum := sum + 1;
            end if;
        end loop;

        if sum >= 13 then
            result <= '1';
        else
            result <= '0';
        end if;
    end process;
end architecture bhv;
```

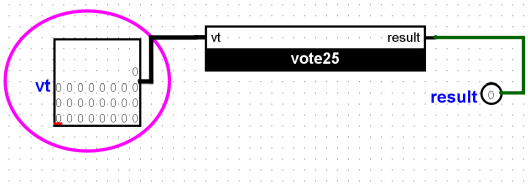
경계 조건 및 대표 사례

찬성 인원	예상 result	캡처 결과
0명	0	0
12명	0	0
13명 (하위 비트)	1	1
13명 (상위 비트)	1	1
25명	1	1

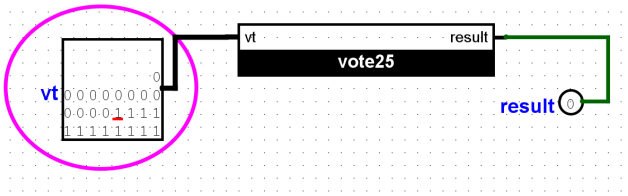
process(vt)에서 입력 25개를 순회하며 1인 비트를 세고, 누적값이 13 이상이면 result를 1로 설정한다. 12명과 13명에서 결과가 바뀌는 과반수 경계를 확인하였다.

투표기 동작 캡처

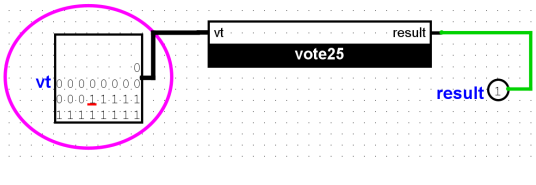
찬성 인원과 1비트 판정 결과를 함께 확인



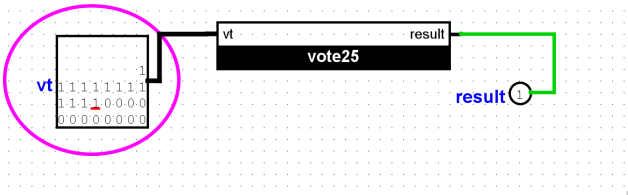
0명 → result 0



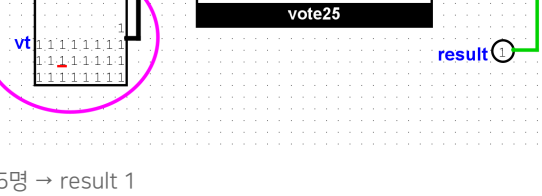
12명 → result 0



13명 → result 1



상위 13명 → result 1



25명 → result 1

특히 비트 위치를 바꾸어도 찬성 개수가 13개면 result=1이 유지되는 것을 확인하였다. 결과는 투표자 위치가 아니라 1의 개수에 의존한다.

13비트 1의 개수 표시

VHDL Entity counter13_seg7 / vt[12:0], en → display[7:0]

VHDL 코드

```
library ieee;
use ieee.std_logic_1164.all;

entity counter13_seg7 is
    port(
        vt      : in std_logic_vector(12 downto 0);
        en      : in std_logic;
        display : out std_logic_vector(7 downto 0)
    );
end entity counter13_seg7;

architecture bhv of counter13_seg7 is
begin
    process(vt, en)
        variable count_ones : integer range 0 to 15;
    begin
        count_ones := 0;

        -- Count the number of 1s
        for i in 0 to 12 loop
            if vt(i) = '1' then
                count_ones := count_ones + 1;
            end if;
        end loop;

        if en = '0' then
            display <= "00000000";
        else
            case count_ones is
                when 0  => display <= "11111100";
                when 1  => display <= "01100000";
                when 2  => display <= "11011010";
                when 3  => display <= "11110010";
                when 4  => display <= "01100110";
                when 5  => display <= "10110110";
                when 6  => display <= "10111110";
                when 7  => display <= "11100000";
                when 8  => display <= "11111110";
                when 9  => display <= "11110110";
                when 10 => display <= "11101110"; -- A
                when 11 => display <= "00111110"; -- b
                when 12 => display <= "10011100"; -- C
                when 13 => display <= "01111010"; -- d
                when 14 => display <= "10011110"; -- E
                when 15 => display <= "10001110"; -- F
                when others => display <= "00000000";
            end case;
        end if;
    end process;
end architecture bhv;
```

입력의 1의 개수를 0~13 범위로 계산하고 case 문으로 7-Segment 출력 패턴을 결정한다. en=0이면 display가 모두 0이 되며, en=1이면 계산 결과를 표시한다. 14·15에 해당하는 E·F 분기도 코드에 포함했지만 실제 13비트 입력에서는 생성되지 않는다.

13비트 표시 동작 확인

1의 개수와 7-Segment 출력, en의 동작

시뮬레이션 결과

vt (설정 상태)	1의 개수	en	7-Segment
전부 0	0	1	0
1이 5개	5	1	5
1이 9개	9	1	9
1이 10개	10	1	A
전부 1	13	1	d
전부 0	0	0	꺼짐

출력 구성

VHDL Entity의 8비트 display를 Splitter로 8개의 1비트 선으로 분할하고, a,b,c,d,e,f,g와 dp에 연결하였다. 본 실험은 Active-high로, 표시할 획만 1을 출력한다.

Enable 동작

en이 1이면 13비트 입력에서 1의 개수를 세어 7-Segment로 표시한다. en이 0이면 동일한 vt 값에 관계없이 display=00000000으로 설정하여 모든 획을 끈다. 따라서 vt=0을 나타내는 숫자 0 표시와 enable에 의한 소등은 서로 다른 동작이다.

문자 A~F 표기 기준

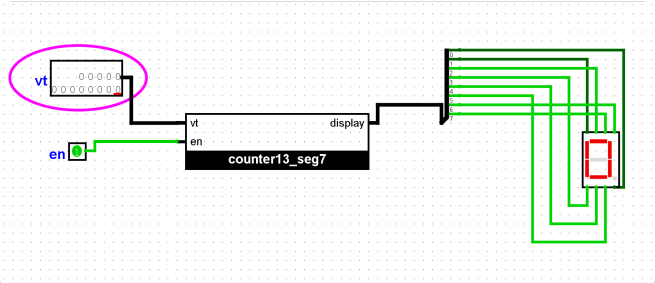
과제 원문은 1의 개수가 10~15일 때 A, B, C, D, E, F를 차례로 표시하도록 요구한다. 본 회로는 이 여섯 값에 대응하는 패턴을 코드에 정의하였다. 단, 7-Segment에서 대문자 B는 숫자 8, 대문자 D는 숫자 0과 구별하기 어려워 16진수 표시에 흔히 쓰는 소문자 모양 b와 d로 표현하였다. 문자에 대응하는 개수 자체는 바뀌지 않는다.

1의 개수	원문 표기	7-Segment 표시	출력 패턴
10	A	A	11101110
11	B	b	00111110
12	C	C	10011100
13	D	d	01111010
14	E	E (도달 불가)	10011110
15	F	F (도달 불가)	10001110

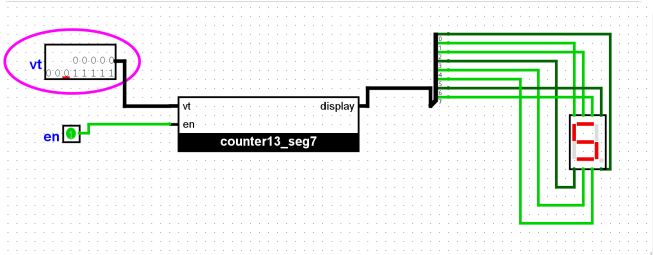
입력 vt는 13비트이므로 1의 개수는 0~13이다. 따라서 14개(E)와 15개(F)는 현재 입력으로 발생하지 않으며, 이 두 분기는 대응 관계를 정의한 코드에만 포함된다.

13비트 표시기 시뮬레이션 캡처

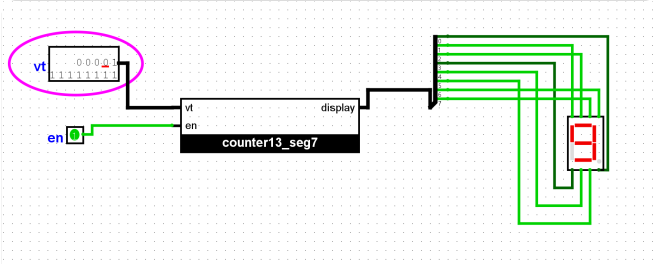
표시값과 Enable OFF 화면



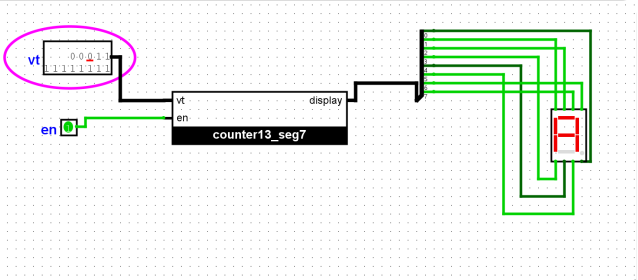
0개 → 숫자 0



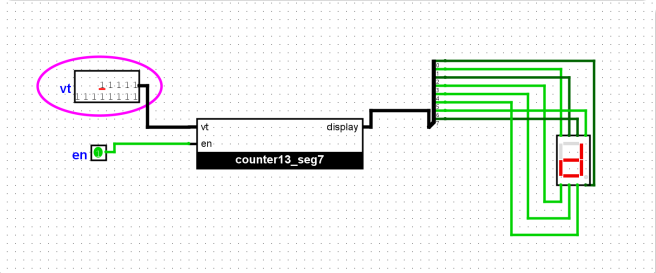
5개 → 숫자 5



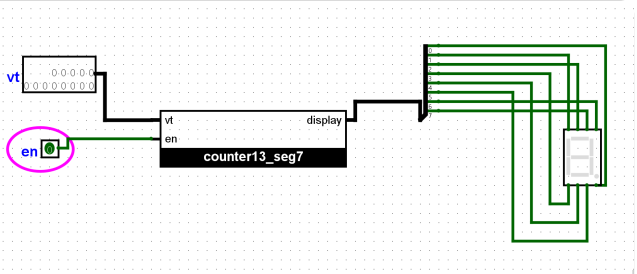
9개 → 숫자 9



10개 → 문자 A



13개 → 문자 d



en=0 → 전체 소등

결과 및 고찰

다섯 가지 회로의 동작 결과와 설계 과정에서 확인한 내용

실험 결과

이번 실습에서는 VHDL을 사용하여 4입력 AND/OR, 8-to-1 Multiplexer, 7-Segment 문자 표시기, 25인 투표기, 13비트 입력의 1개수 표시기를 구현하였다. 각 설계를 Logisim-evolution의 VHDL Entity로 불러와 입력값을 직접 바꾸면서 출력이 논리식 및 코드의 조건과 일치하는지 확인하였다.

1번에서는 abcd=0000일 때 두 출력이 모두 0, 1110일 때 AND 출력은 0이고 OR 출력은 1, 1111일 때 두 출력이 모두 1로 나타났다. 이를 통해 AND는 모든 입력이 1이어야 출력이 1이 되고, OR는 하나 이상의 입력이 1이면 출력이 1이 된다는 점을 확인하였다. 2번의 8-to-1 MUX는 입력 a~h를 10101010으로 설정하고 ch를 변경하였다. ch가 000, 001, 110, 111일 때 출력은 각각 1, 0, 1, 0으로 나타나 선택 신호에 해당하는 입력이 출력으로 전달되었다.

3번에서는 3비트 sel에 따라 a, b, c, d, f, g에 해당하는 7-Segment 패턴이 표시되는 것을 확인하였다. 4번의 25인 투표기는 찬성표 12개일 때 result=0, 13개일 때 result=1을 보여 과반수 판정 기준을 만족하였다. 찬성표를 상위 비트 쪽에 배치한 경우에도 찬성 인원이 13명이면 결과가 1이었으며, 전체 반대와 전체 찬성의 출력도 각각 0과 1로 확인하였다.

5번은 13비트 vt에서 1의 개수를 계산하여 7-Segment에 표시하였다. 대표적으로 1이 0개, 5개, 9개, 10개, 13개일 때 숫자 0, 5, 9와 문자 A, d가 표시되었다. en을 0으로 두었을 때는 입력값과 관계없이 표시가 꺼졌으며, en이 1인 상태에서 vt의 1의 개수가 0일 때 숫자 0이 표시되는 경우와 구분할 수 있었다.

고찰

앞의 1번과 2번은 불대수 연산 및 데이터 선택 기능을 비교적 단순한 입출력 관계로 기술할 수 있었다. 반면 4번과 5번은 여러 입력 비트를 검사하고 1의 개수를 누적해야 하므로, process 안에서 변수를 초기화한 뒤 for 문으로 반복 계산하는 방법을 사용하였다. 이 방식은 입력 비트 수가 늘어나더라도 논리식을 일일이 전개하지 않고 동작을 표현할 수 있다는 장점이 있었다.

특히 4번 투표기에서는 단순히 찬성표가 많다는 사실보다 정확한 경계 조건을 확인하는 것이 중요했다. 25명의 절반은 12.5명이므로 12명과 13명에서 출력이 달라져야 한다. 또한 1의 위치가 달라져도 개수가 같다면 출력 역시 같아야 하는데, 상위 비트와 하위 비트에 찬성표를 배치한 사례에서 이를 확인하였다.

7-Segment를 사용한 3번과 5번에서는 출력 비트가 실제로 어느 선분에 연결되는지가 중요했다. 8비트 display의 각 비트와 a~g, dp의 연결 순서를 맞추어야 코드에서 정의한 패턴을 의도한 모양으로 표시할 수 있었다. 또한 5번은 원문에서 A~F를 요구하지만, 7-Segment 특성상 대문자 B와 D는 각각 숫자 8과 0에 가까워지므로 일반적인 16진수 표시 방식인 b와 d로 나타냈다. 이는 문자에 대응하는 수치가 아니라 선분 모양의 차이이다.

마지막으로 5번의 입력은 13비트이므로 1의 개수가 14개 또는 15개가 되는 경우는 존재하지 않는다. 따라서 E와 F의 출력 패턴은 코드에 정의했지만 실제 vt 입력으로 확인할 수 있는 범위는 0~13이며, A~D에 해당하는 개수까지만 도달한다. 이번 실습을 통해 논리 기능을 작성하는 것뿐 아니라 입력 범위, 경계 조건, 출력 장치의 표현 한계를 함께 고려해야 한다는 점을 확인하였다.